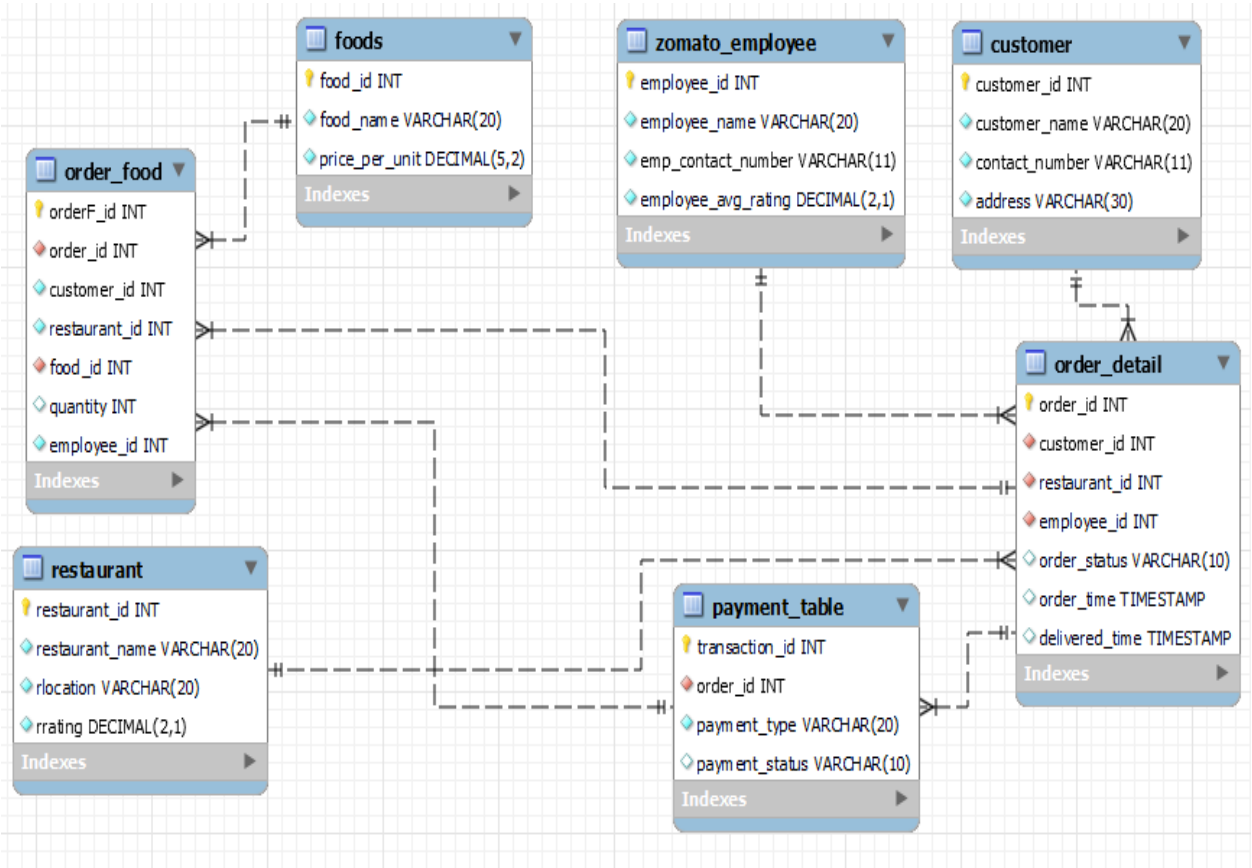
Zomato Sales & Operational Analysis using SQL

Author: Anjul

This project demonstrates SQL-based business analysis using multi-table joins, subqueries, aggregation, and time-based functions.

Database Schema (ER Diagram)



Q15: Revenue Contribution by Payment Type

```sql
-- Q15: Total Revenue Contribution by Payment Type
SELECT pt.payment_type,
SUM(order_total_amount) AS total_payment_type
FROM payment_table pt
JOIN
(
    SELECT ofd.order_id,
    SUM(ofd.quantity * f.price_per_unit) AS order_total_amount
    FROM order_food ofd
    JOIN foods f ON ofd.food_id = f.food_id
    GROUP BY ofd.order_id
) AS order_totals
ON pt.order_id = order_totals.order_id
GROUP BY pt.payment_type
ORDER BY total_payment_type DESC;

-- Insight: Internet Banking contributes the highest revenue (₹6535),
followed by COD (₹3370), indicating strong customer preference
-- toward digital_payment methods.
```

Result Grid | Filter Rows:

payment_type	total_payment_type
internet banking	6535.00

Q1: Top 3 Customers by Total Orders

```sql
-- Q1: Top 3 Customers by Total Orders

SELECT customer_id, COUNT(order_id) AS total_orders
FROM order_detail
GROUP BY customer_id
ORDER BY total_orders DESC
LIMIT 3;

-- Insight: Customer 1001 is the most frequent customer with 11 orders.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	total_orders
1001	11
1008	4
1003	4

Q4: Revenue by Food Item

```sql
-- Q4: Total Revenue Generated by Each Food Item

SELECT f.food_id, f.food_name,
SUM(ofd.quantity * f.price_per_unit) AS total_revenue
FROM order_food ofd
JOIN foods f ON ofd.food_id = f.food_id
GROUP BY f.food_id, f.food_name
ORDER BY total_revenue DESC;

-- Insight: Identifies the highest revenue-generating food items,
-- helping understand product-level performance.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

food_id	food_name	total_revenue
9999416	veg biryani	2520.00
9999419	mix paratha	1620.00
9999417	mix veg	900.00
9999422	veg roll	900.00
9999411	allo paratha	880.00
9999420	cold drink	665.00
9999414	veg thali	660.00
9999412	veg meal	480.00

Result 18

Q5: Second Highest Revenue-Generating Restaurant

```
-- Q5: Second Highest Revenue-Generating Restaurant

SELECT r.restaurant_name,
SUM(ofd.quantity * f.price_per_unit) AS total_revenue
FROM order_food ofd
JOIN foods f ON ofd.food_id = f.food_id
JOIN order_detail od ON ofd.order_id = od.order_id
JOIN restaurant r ON od.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_name
ORDER BY total_revenue DESC
LIMIT 1 OFFSET 1;

-- Insight: Identifies the second best-performing restaurant
-- based on total revenue contribution.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

restaurant_name	total_revenue
laa unico	1675.00

Q6: Top 5 Most Popular Food Items

```sql
-- Q6: Top 5 Most Popular Food Items Based on Quantity Sold

SELECT f.food_id, f.food_name,
SUM(ofd.quantity) AS total_quantity_sold
FROM order_food ofd
JOIN foods f ON ofd.food_id = f.food_id
GROUP BY f.food_id, f.food_name
ORDER BY total_quantity_sold DESC
LIMIT 5;

-- Insight: Highlights the most in-demand food items,
-- useful for inventory and marketing decisions.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

food_id	food_name	total_quantity_sold
9999420	cold drink	19
9999413	chapati	12
9999411	allo paratha	11
9999419	mix paratha	9
9999416	veg biryani	9

Result 20

Q8: Peak Order Month Analysis

Limit to 1000 rows

```sql
-- Q8: Month with Highest Total Number of Orders

SELECT MONTH(order_time) AS month,
COUNT(order_id) AS total_orders
FROM order_detail
GROUP BY MONTH(order_time)
ORDER BY total_orders DESC
LIMIT 1;

-- Insight: Identifies peak order month,
-- helping understand seasonal demand trends.
```

Result Grid | Filter Rows: | Export: | Wrap Ce

month	total_orders
10	38

Q9: Average Order Amount per Customer

```sql
-- Q9: Average Order Amount for Each Customer

SELECT od.customer_id,
AVG(ofd.quantity * f.price_per_unit) AS avg_order_amount
FROM order_detail od
JOIN order_food ofd ON od.order_id = ofd.order_id
JOIN foods f ON ofd.food_id = f.food_id
GROUP BY od.customer_id
ORDER BY avg_order_amount DESC;

-- Insight: Measures customer spending behavior
-- and identifies high-value customers.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	avg_order_amount
1009	320.000000
1013	280.000000
1014	246.666667
1008	241.000000
1011	197.500000

Result 22

Q12: Average Delivery Time (Weekday vs Weekend)

```sql
-- Q12: Average Delivery Time Comparison (Weekday vs Weekend)

SELECT
CASE
    WHEN DAYOFWEEK(order_time) IN (1,7) THEN 'Weekend'
    ELSE 'Weekday'
END AS day_type,
AVG(TIMESTAMPDIFF(MINUTE, order_time, delivered_time)) AS avg_delivery_time
FROM order_detail
GROUP BY day_type;

-- Insight: Compares delivery efficiency between weekdays and weekends,
-- useful for operational performance analysis.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

weekend_orders
13